

KIRO EN MODO CONSTRUCTOR

specs, agentes y una web real

Manual paso a paso

Mi tercer proyecto: KDD

Ejercicio relámpago con Kiro (IDE de AWS) — fase "Crear"

CONTENIDO DE ESTE MANUAL

- 01 Identificas el conocimiento del proyecto
- 02 Escribes el prompt del archivo de steering
- 03 Verificas que quede en .kiro/steering/
- 04 Kiro carga el steering automáticamente
- 05 Un prompt neutro sale on-brand, sin pedirlo
- 06 Revisas el detalle de lo que construyó
- 07 SDD vs. KDD: qué pregunta responde cada uno

Sábado 12 de septiembre de 2026 · 9:00 - 11:00 a.m.

UNAB · Salón L51 · Bucaramanga

Organizan: BucaraTec · AWS User Group Bucaramanga · UNAB TEC

¿Qué vamos a hacer?

Este manual documenta el tercer enfoque del ejercicio relámpago del taller "Kiro en modo constructor: specs, agentes y una web real": Knowledge-Driven Development (KDD). A diferencia de Vibe Coding (un prompt, sin memoria) y de SDD (un spec por funcionalidad), aquí el conocimiento del proyecto —su marca, en este caso— se escribe una sola vez y queda disponible para cualquier tarea futura.

El objetivo es ver de principio a fin cómo se crea un archivo de steering, cómo Kiro lo carga automáticamente antes de construir nada, y cómo el mismo tipo de prompt que en Vibe Coding salía sin marca, aquí sale on-brand sin pedirlo.

PASO 1

Identificas el conocimiento del proyecto

Antes de escribir un solo prompt de construcción, decides qué debe "saber siempre" este proyecto: la marca, el stack tecnológico, los patrones de código, las convenciones del equipo. Eso es lo que va a vivir en un archivo de steering.

Se guarda en markdown dentro de `.kiro/steering/` —por ejemplo `product.md`, `structure.md`, `tech.md`, o uno propio como `brand.md`— y se genera con "Generate Steering Docs" o se escribe a mano.

A diferencia de un spec de SDD, que resuelve qué construir esta vez, un archivo de steering resuelve qué debe saber el proyecto siempre, sin que se repita en cada prompt.

PASO 2

Escribes el prompt del archivo de steering

Se le pide a Kiro que cree el archivo directamente, con el contenido completo de la guía de marca ya definido:

Crea un archivo de steering en `.kiro/steering/brand.md` con este contenido, para que se aplique automáticamente a cualquier proyecto web que construya de aquí en adelante, sin que yo tenga que repetirlo:

```
# Marca: AWS User Group Bucaramanga

## Colores (usar solo esta paleta, no combinar con otros)
- Dark Squid Ink: #161E2D (fondo principal)
- Morado: #A166FF (acentos, títulos clave, botones)
- Blanco: #FFFFFF (texto principal sobre fondo oscuro)

## Tipografía
- Arial para todo el contenido (títulos, cuerpo, botones)
- Amazon Ember solo aparece dentro del logo – nunca se usa como
fuente del sitio ni se reemplaza en el logo

## Estilo visual
- Fondo oscuro (Dark Squid Ink) como base; el morado es acento,
no protagonista
- Motivo hexagonal como elemento decorativo (línea, no relleno)
- Nada de colores fuera de esta paleta, nada de gradientes con
otros tonos

## Logos
- El logo de AWS User Group Bucaramanga va al inicio/hero de
cualquier página del evento
- Los logos de los organizadores van en una fila, en la apertura
o el cierre de la página
- Nunca alterar, recolorear ni deformar ningún logo

Aplica esto a cualquier página, landing o interfaz que construyas
para este proyecto, sin que yo lo pida en cada prompt.
```

Lo que define este archivo:

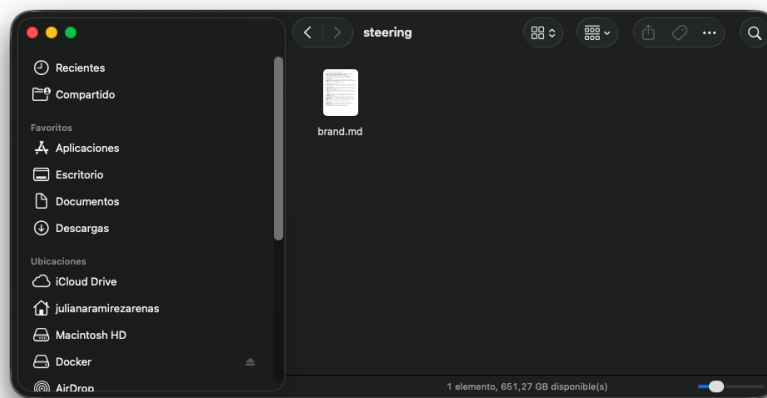
- Una paleta cerrada de 3 colores, sin mezclar con otros
- Arial como única fuente del sitio; Amazon Ember solo dentro del logo
- Un motivo hexagonal decorativo, como eco visual del logo
- Reglas de posicionamiento de logos, y la prohibición de alterarlos

Presiona Enter y espera mientras Kiro escribe el archivo ("Working...").

PASO 3

Verificas que quede en `.kiro/steering/`

Confirmas que el archivo haya quedado dentro de la carpeta correcta del proyecto:



Carpetas del proyecto con brand.md dentro de steering/

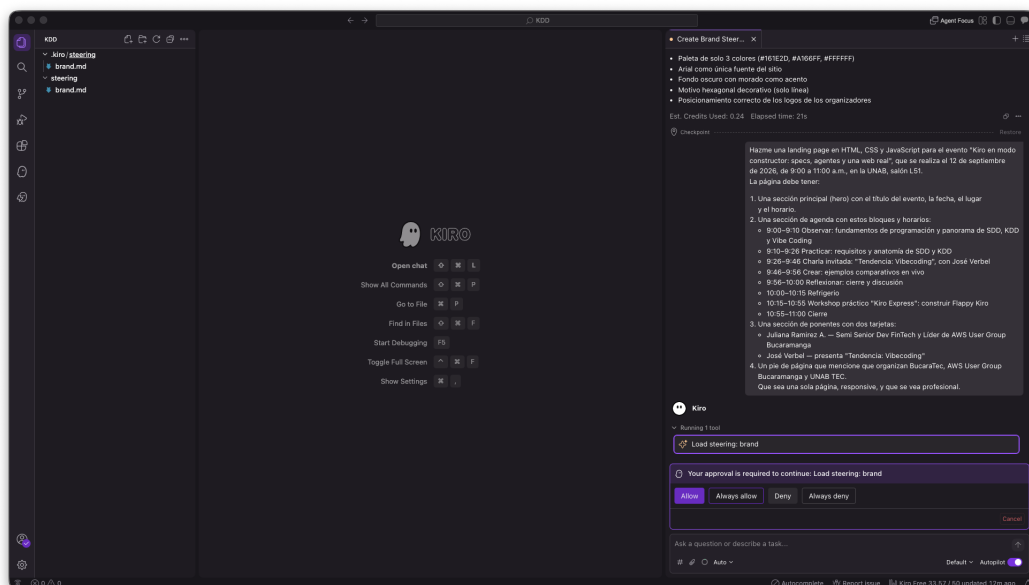
El archivo vive dentro del proyecto —no en el chat, no en la memoria de una conversación— así que sobrevive a cerrar Kiro, cambiar de computador o de persona.

Lo que no se ve en Finder es lo que de verdad importa: las primeras líneas del archivo deben ser un bloque --- con inclusion: always, sin nada antes. Sin ese front matter, Kiro no carga el archivo en cada tarea —hay que abrirlo y confirmarlo antes de seguir.

PASO 4

Kiro carga el steering automáticamente

Corres el primer prompt de construcción en este proyecto, y antes de tocar un solo archivo, Kiro pide permiso para algo que nadie le pidió:



"Load steering: brand" — ejecutado sin que el prompt lo mencionara

Al correr un prompt de construcción en este proyecto, ocurre algo que no pasó en el ejercicio de Vibe Coding: antes de escribir una sola línea, Kiro ejecuta la herramienta Load steering: brand. Nadie se lo pidió en el prompt — es la prueba de que el archivo se está consultando automáticamente.

Cargar el steering requiere tu aprobación, con cuatro opciones:

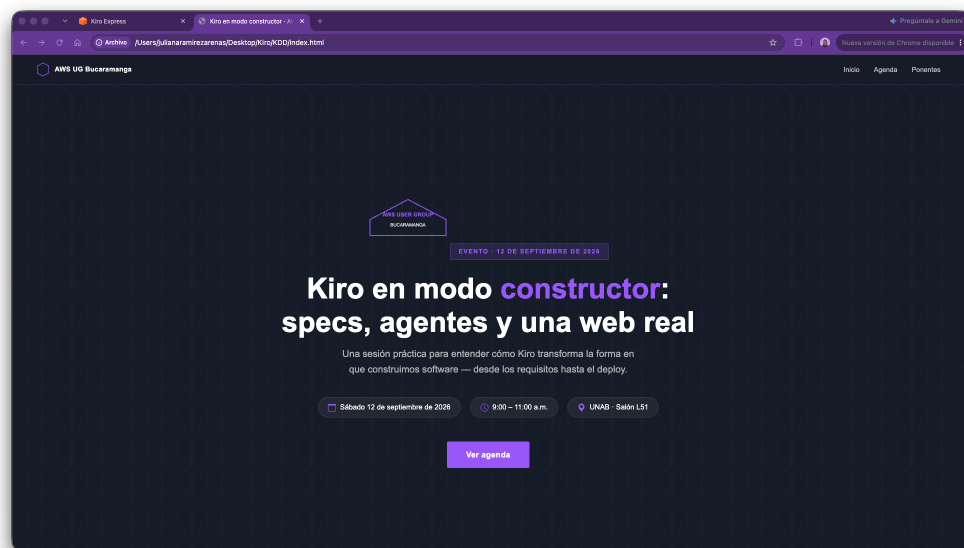
- Allow — aprueba esta carga puntual, una sola vez
- Always allow — aprueba este tipo de carga automáticamente de ahí en adelante
- Deny — rechaza esta carga puntual
- Always deny — bloquea este tipo de carga de forma permanente

"Always allow" evita que la aprobación interrumpa el flujo si el ejercicio se repite varias veces antes de una demo en vivo.

PASO 5

Un prompt neutro sale on-brand, sin pedirlo

Tras aprobar los cambios, Kiro abre la página en el navegador:



Resultado renderizado en el navegador

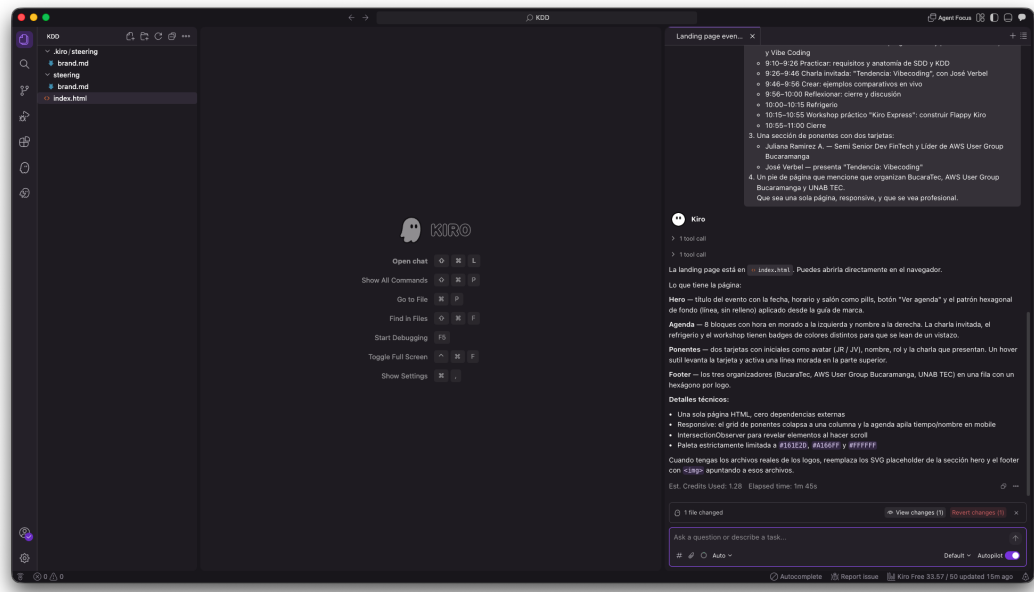
El prompt que generó esta página solo pedía una sección hero con título, fecha, lugar y horario — no mencionó colores, tipografía ni ningún motivo decorativo. Aun así, el resultado aplicó Dark Squid Ink de fondo, el morado como acento en "constructor" y en el botón, y un patrón hexagonal de fondo que hace eco del logo.

Esa es la diferencia con Vibe Coding: allá la marca salió porque el prompt la pidió explícitamente ("usa los colores de la marca, los logos..."); aquí salió porque el proyecto ya la sabía de antemano.

PASO 6

Revisas el detalle de lo que construyó

Le pides a Kiro que resuma lo que acaba de construir, sección por sección:



Kiro describe la página construida, sección por sección

- Hero — título con fecha, horario y salón como pills, botón "Ver agenda" y el patrón hexagonal de fondo aplicado desde la guía de marca
- Agenda — bloques con hora en morado y badges de color para diferenciar la charla invitada, el refrigerio y el workshop
- Ponentes — tarjetas con iniciales como avatar, y una línea morada que se activa al pasar el cursor
- Footer — los tres organizadores en fila, con un hexágono como remate visual
- Detalles técnicos — una sola página HTML, responsive, y la paleta estrictamente limitada a los tres colores del steering

Cuando el logo real de AWS User Group Bucaramanga esté disponible en la carpeta assets del proyecto, basta con pedirle a Kiro que lo use en el hero y en el footer, en lugar del marcador hexagonal que generó por su cuenta.

SDD vs. KDD: qué pregunta responde cada uno

No es que uno sea mejor que el otro; cada uno responde una pregunta distinta.

	SDD	KDD
Responde a	"¿Qué voy a construir esta vez?"	"¿Qué debe saber el proyecto siempre?"

	SDD	KDD
Alcance	Una funcionalidad puntual	Todo el proyecto, cualquier funcionalidad futura
Dónde vive	Dentro de un spec: requirements.md → design.md → tasks.md	En .kiro/steering/, fuera de cualquier spec
Una vez, cuando actúa	Cuando defines o construyes algo nuevo	De fondo, en cada tarea, sin que lo invoques
Se combina con	Con steering: ya sabe la marca antes de escribir el spec	Con Vibe y con SDD por igual

EN RESUMEN: LAS 4 IDEAS CLAVE

- El steering no resuelve una tarea puntual: resuelve el "siempre" del proyecto — la marca, el stack, las convenciones.
- Queda versionado junto al código, así que sobrevive a cerrar el chat, cambiar de computador o de persona.
- Kiro lo consulta en cada tarea, sea Vibe Coding o SDD, sin que lo invoques.
- El resultado sale alineado porque el proyecto ya lo sabía — no porque se pidió esa vez.